Unified integer overflow arithmetic

Document Number:	P3161R5
Date:	2025/03/24
Reply-to:	cpp@kaotic.software
Authors:	Tiago Freire
Audience:	SG6, LWG

Target
C++29

Abstract

Addition and uniformization of integer arithmetic functions with overflow behavior

Revision

#	Description
0	Initial draft
1	Corrected suggested implementation of would_cast_modify, corrected div_wide definition, and made minor editorial changes.
2	Removed would_cast_modify in favor of existing in_range, added some code examples
3	Removed incorrect number of template parameters in wording. Added section of suggested optimizations and reference implementation.
4	Dropped feature test macro substitution. Updated wording to reflect latest draft. Other miscellaneous wording corrections.
5	Re-written paper structure. Dropped signed div_wide, is_div_wide_defined, and is_div_defined functions.

Table of Contents

1. Motivation

In many applications sometimes one has to deal with integer arithmetic with numbers whose width (bit count) far exceeds that for which the defined standard types can support, or ever will be able to support in the since the width of the representable numbers are application-specific and can grow arbitrarily large (within the finite capabilities of the underlying device). In other applications even if extended precision is not required it might be important to know if the result of an operation is valid (i.e. that it did not overflow).

Algorithms to deal with these are quite trivial, and most CPUs offer a good range of support wwith dedicated instructions (with these exact usages in mind), but there are no equivalent abstractions available in the standard C++. It's rather cumbersome and error prone to implement similar functionality using only C++, resulting in extremely inefficient code for what could often be a couple or even a single line of assembly.

The paper [P0543], has already been accepted and is on track for C++26, however "saturation" is just a narrow type of overflow behavior, which is insufficient to implement things like multi-word integers.

Taking into consideration that:

1. Some multi-word operations require a combination of multiple instances of such functions for a specific type.
2. CPUs (such as x86-64 or arm64) may require certain operands to be preloaded into specific registers, and this is typically achieved by **mov** instruction for the preparation of parameters and recovery of the result.
3. And in many of these cases the output register just so happens to be perfectly aligned with the register where the value needs to be on a follow up operation, making a **mov** instruction unnecessary.

An optimizer aware of this, that is free to re-order independent operations and freely swap commutative operands (such as addition and multiplication) can maximize the suppression of mov instructions, and this is something that is best performed at the compiler level which has a holistic view of the structure and context of the code being generated. An independent library writer that has only view of the function being written cannot do this, but a team working closer to the compiler development knowing that there is a guaranteed standard way to represent this can. Thus, given the nature and importance of these algorithms, it is extremely important for these functions to be standardized.

2. Logical organization

The integer arithmetic function featured in this paper can be organized in the following way:

They can be divided in terms of related operations

- addition
- subtraction
- multiplication
- division/remainder - reciprocal operation to multiplication
- casting - The conversion from one machine representation of an integer type to another

Or divided in terms of families of overflow behavior:

- Standard - No explicit overflow behavior (besides wrap-around), these are the standard C++ operations (+, -, *, /, %)
- Saturation - Value saturates on overflow, as adopted by paper [P0543]
- Reporting - Operations reports when an overflow occurs

- Extended width - Output of operation has extended width to avoid overflow

The logical grouping of functions can be summarized by the following table:

Type of operation	Standard	Saturated	Safe overflow	Wide arithmetic
Addition	+	add_sat		add_carry
Subtraction	-	sub_sat		sub_borrow
Multiplication	*	mul_sat		mul_wide
Division, Remainder	/, %, div	div_sat	is_div_defined	div_wide, is_div_wide_defined
Type casting	static_cast	saturate_cast	in_range	N/A

3. List of functions

3.1. add_carry

Suggested signature

```
template<class T>
constexpr add_carry_result<T> add_carry(T, T, bool carry) noexcept;
```

CPU support

x86-64	ARM64
ADC, ADCX, ADOX, ADD	ADC, ADCS, ADDS

Non-portable precedent

clang	msvc	intel intrinsic
__builtin_addcb, __builtin_addcs, __builtin_addc, __builtin_addcl, __builtin_addcll	-	_addcarry_u8, _addcarry_u16, _addcarry_u32, _addcarry_u64, _addcarryx_u32, _addcarryx_u64

Description:
Addition with carry
With *T* an integer input type, and inputs:

- V1 :T
- V2 :T
- carry :bool

Performs the following operation as if by unlimited precision:
result = V1 + V2 + (carry ? 1 : 0);
and outputting:

- low_result :T - the low bits of *result* with the same width of T
- overflow :bool - a bool flag that is set to true if *low_result* does not represent the value of *result* (i.e. overflow)

Hint: CPU instruction can be converted to an ADD/ADDS if the carry bit can be determined at compile time to be 0.

3.2. sub_borrow

Suggested signature

```
template<class T>
constexpr sub_borrow_result<T> sub_borrow(T minuend, T subtrahend, bool
borrow) noexcept;
```

CPU support

x86-64	ARM64
SBB, SUB	SBC, SBCS, SUBS

Non-portable precedent

clang	msvc	intel intrinsic
__builtin_subcb, __builtin_subcs, __builtin_subc, __builtin_subcl, __builtin_subcll	-	_subborrow_u8, _subborrow_u16, _subborrow_u32, _subborrow_u64

Description:
Subtraction with borrow
With *T* an integer input type, and inputs:

- minuend :T
- subtrahend :T
- borrow :bool

Performs the following operation as if by unlimited precision:
result = minuend - subtrahend - (borrow ? 1 : 0);
and outputting:

- low_result :T - the low bits of *result* with the same width of T
- overflow :bool - a bool flag that is set to true if *low_result* does not represent the value of *result* (i.e. overflow)

Hint: CPU instruction can be converted to an SUB/SUBS if the carry bit can be determined at compile time to be 0.

3.3. mul_wide

Suggested signature

```
template<class T>
constexpr mul_wide_result<T> mul_wide(T, T) noexcept;
```

CPU support

x86-64	ARM64
IMUL, MUL, MULX	SMULL, UMULL, (MUL, SMULH, UMULH)

Non-portable precedent

clang	msvc	intel intrinsic
-	_mul128, _umul128, __emul, __emulu, __mulh, __umulh	mulx_u32, mulx_u64

Description:
Multiplication with twice the width of inputs. With *T* an integer input type, and inputs:

- V1 :T
- V2 :T

Performs the following operation as if by unlimited precision:
result = V1 * V2;
and outputting:

- low_result :T - the low-bits of *result*

- high_result :T - the high-bits of *result*

3.4. div_wide

Suggested signature

```
template<class T>
constexpr div_result<T> div_wide(T dividend_high, T dividend_low, T
divisor) noexcept;
```

CPU support

x86-64	ARM64
DIV, IDIV	-

Non-portable precedent

clang	msvc	intel intrinsic
-	_udiv64, _udiv128, _div64, _div128,	-

Description:
The reciprocal operation to mul_wide With *T* an integer input type, and inputs:

- dividend_high :T
- dividend_low :T
- divisor :T

Performs the following operation as if by unlimited precision:
dividend = (dividend_high << NBT) bitor dividend_low;
result_quo = dividend / divisor;
result_rem = dividend % divisor;
where "<< NBT" is a bitswise left shift by the number of bits of type T
and outputting:

- output_quo :T - *result_quo* truncated to the width of T
- output_rem :T - *result_rem*

Note:

- If divisor is 0 or if output_quo overflows the behavior is undefined. Undefined behavior can be pre-checked with *!is_div_wide_defined*
- result_quo is the value such that abs(result_quo) has the lowest possible value that satisfies abs(dividend - divisor * result_quo) < abs(divisor)

3.5. is_div_defined

Suggested signature

```
template<class T>
constexpr bool is_div_defined(T dividend, T divisor) noexcept;
```

Description:
Checks if std::div is defined for the input arguments
i.e. Checks that the divisor is not 0, and in case of signed division that result would not overflow, i.e. inputs are not the degenerate case std::numeric_limits::min() / -1.

Possible implementation:

```
template<typename T>
[[nodiscard]] inline constexpr bool is_div_defined([[maybe_unused]] T const
dividend, T const divisor) noexcept
{
    if constexpr(std::is_signed_v<T>)
    {
        return divisor != 0 && (dividend != std::numeric_limits<T>::min()
|| divisor != -1);
    }
    else
    {
        return divisor != 0;
    }
}
```

Note:
Given how trivial it is to write this function, this paper does not propose adding it to the standard at this time.

3.6. is_div_wide_defined

Suggested signature

```
template<class T>
constexpr bool is_div_wide_defined(T dividend_high, T dividend_low, T
divisor) noexcept;
```

Description:
Checks if std::div_wide is defined for the input arguments
i.e. Checks that the divisor is not 0, and that result would not overflow

Possible implementation:
See Annex B.2.

4. Examples

The following are examples of algorithms that could be written using the suggested additions in this paper.
Presented for illustrative purposes.

4.1. Signed Big number addition

```
struct SignedBigNum
{
    using uint_t = uint64_t;
    using sint_t = int64_t;

    std::vector<uint_t> low_bits;
    sint_t high_bits;
};

SignedBigNum BigNum_same_size_Add(SignedBigNum const& v1, SignedBigNum
const& v2)
{
    using uint_t = SignedBigNum::uint_t;
    using sint_t = SignedBigNum::sint_t;

    uintptr_t const size = v1.low_bits.size();
    assert(size == v2.low_bits.size());

    SignedBigNum temp_out;
    temp_out.low_bits.resize(size);

    bool overflow = false;

    for(uintptr_t i = 0; i < size; ++i)
    {
        //unsigned add with carry
        std::add_carry_result<uint_t> const temp_result =
std::add_carry<uint_t>(v1.low_bits[i], v2.low_bits[i], overflow);
        temp_out.low_bits[i] = temp_result.low_result;
        overflow = temp_result.overflow;
    }

    //signed add with carry
    std::add_carry_result<sint_t> const temp_result =
std::add_carry<sint_t>(v1.high_bits, v2.high_bits, overflow);
    if(temp_result.overflow)
    {
        temp_out.low_bits.push_back(static_cast<uint_t>
(temp_result.low_result));
        temp_out.high_bits = static_cast<sint_t>
(get_sign_bit(temp_result.low_result) ? const_0_pad : const_1_pad);
    }
    else
    {
        temp_out.high_bits = temp_result.low_result;
        //remove unnecessary bits (optional)
    }

    return temp_out;
}
```

4.2. Extract from algorithm calculating all base 10 digits of a floating-point number

```
//uint64_t carry;
for(uint8_t i = 1; i < last_index; ++i)
{
    if(!p_1[i])
    {
        p_1[i] = carry;
        return;
    }

    //could have been simplified with fused multiply add
    auto mul_res = std::mul_wide(p_1[i], p_2);
    auto add_res = std::add_carry(0, mul_res.low_bits, carry);
    if(add_res.overflow)
    {
        ++mul_res.high_bits;
    }

    if(mul_res.high_bits || add_res.low_bits >= fp_utils_p::max_pow_10)
    {
        auto div_result = std::div_wide(mul_res.high_bits,
add_res.low_bits, fp_utils_p::max_pow_10);
        carry = div_result.quotient;
        p_1[i] = div_result.remainder;
    }
    else
    {
        p_1[i] = add_res.low_bits;
        carry = 0;
    }
}
```

4.3. Unsigned Big number division by simple scalar

```
struct UnsignedBigNum
{
    using uint_t = uint64_t;
    std::vector<uint_t> bits;
};

UnsignedBigNum BigNum_divide(UnsignedBigNum const& v1, uint64_t const v2)
{
    using uint_t = UnsignedBigNum::uint_t;
    if(v2 == 0)
    {
        return {};
    }

    uintptr_t const size = v1.bits.size();

    UnsignedBigNum temp_out;

    temp_out.bits.resize(size);

    std::div_result<uint_t> temp_res {.quotient = 0, .remainder = 0};
    for(uintptr_t i = size; i--;)
    {
        temp_res = std::div_wide(temp_res.remainder, v1.bits[i], v2);
        temp_out.bits[i] = temp_res.quotient;
    }

    return temp_out;
}
```

5. Special considerations for division

All functions in this paper have well defined behavior regardless of the input with the exceptions of those related to division. Undefined behavior occurs when either dividing by zero or when the resulting value would overflow. Functions such as the trivial division (/), std::div, and std::div_sat already expects the user to check and avoid calling the function if it would trigger undefined behavior.

Platforms that need to implement such function in software can decide to inflict only mild consequences upon the user. But since the intention is to utilize specialized CPU instructions, and those instructions trap in these conditions, it is expected that division in practice will have that behavior.

An unhandled trap results in a panic by the operation system which promptly ejects the application from running. While most cases are trivial to check, and an average user would be able to write checks by themselves, signed div_wide on the other hand is not trivial, and I would not expect the average user to so easily be able to write it. Providing such functions without the means to check for undefined behavior would be irresponsible, we wouldn't so much be providing a useful function to implement algorithms, as we would be providing a function that would sporadically and ungracefully eject the user's code without much in the way a user can do to prevent it.

The question then poses, should the functions themselves always check the inputs before attempting the division proper? The answer here is no. Many algorithms can guarantee that no undefined behavior is ever triggered by the way that they are setup without the need to check, and we would be unjustifiably penalizing such users with unneeded overhead. Take for a example an algorithm that tries to reduce a multi-word unsigned number by dividing it with a divisor that is a compile time constant not equal to 0. Such an algorithm would start with an std::div of the highest order word, and then feeding the remainder (which is guaranteed < divisor) as the dividend_high of subsequent calls to div_wide to reduce lower order words, at no point in such an algorithm would it ever be possible to trigger undefined behavior (see 4.3).

While unsigned dive_wide is useful and has many examples of algorithms that require it, the signed dive_wide has not seen wide spread usage. Given the limited usefullness of signed dive_wide, and that it would require a validation function (see is_div_wide_defined) whose current implementation is quite awkward, plus some additional type confusion that would be associated with its implementation (see FAQ), I decided that it would be better if signed div_wide would not be part of this proposal, leaving this question for a future paper if a proper use case is found to justify it.

6. Design choice analysis

6.1. Library header

I agree with the approach presented by [P0543] which adds the new functionality to the <numeric> library.

These are functions that perform numerical operations, a new header is not required.

6.2. Overloads, Function Templates, or Named Functions

The use of template arguments is preferable over the alternatives for the following reasons:

- In contexts where we are working with template types, it would be much easier to just use the name of the function and let the type system figure out which "specialization" to use based on the types being provided. This excludes named functions as it would require creating specific overloads depending on type being used or create my own facility that wraps around these types that do the same job. If I have to create my own facility, and that is what actually gets used in replacement of the standard, then why not make that the standard?
- Using specific names for each type can be error prone, as one not only needs to be careful and specific with the type selection within the context being used. Which increases the risk of picking the wrong type, or having input types not match, thus leading to the silent promotion of the input types without warning and with unintended con sequences. If input types are not consistent it should definitely be a compiler error, and if they disagree the user should be forced to explicitly cast the inputs to the intended type before proceeding.
- There's not a 1 to 1 correspondence between all integer types and all types of bit-width that integers can have (a more detailed explanation below), and not all types of integers are mandatory to be available in all platforms. Just stating in the standard that "the input parameter is a templated type", and that "the only allowed templated types must be integers" is much simpler in order to ensure that all intended cases are covered. A named or overloaded definition would require a disclaimer regarding which integers are supported and which versions are available depending on the combination of integers available for that platform.
- If you require a specific instance of the function for a specific type to be explicitly declared this cannot be easily done with overloads, but can still be easily done with templates as one can just explicitly declare the exact templated type and this happens in a much more natural language e.x. mul_wide<uint32_t> is much more explicit than ui32_mul_wide.

6.3. Return type

There are several ways to return multiple output values. For the cases where there's only 1 output value, just return the value as is with no additional structure. For the other cases these were the options considered:

- Using an existing data structure other than a tuple, such as std::pair
- Using a new dedicated data structure
- Use a tuple

In my opinion it is best to stay away from using types like std::pair given their confusing ambiguity, take for example "mul_wide" return a std::pair assigned to a variable named val, what would be the meaning of "val.first" or "val.second"? Do you put the high bits in ".first" or ".second"? Unless the names of the member variables

explicitly spell their meaning (ex. result_high and result_low) the whole thing is just hard to read, a dedicated data structure has better properties for this purpose.

If we use a new dedicated data structure, now the problem is what do we name them? They are relatively specific to the function that they are associated with, so one could for example use the pattern <function_name>_result (similar to what happens with std::from_chars), example "mul_wide_result".
In addition, the type would need to be templated since the function parameters and consequentially the types of the expected output is also templated. This would require defining at least 4 new templated data types, in most cases supporting 10 different integer types, for a potential of 40 new data structures in practice. This isn't a big deal, compilers can certainly handle that quite easily, and the effort for implementing those is proportional to the effort of implementing the new features, but can we do better?

I lament the fact that C++ doesn't have a better syntax and support features for multiple return values. std::tuple has become the closest thing to it, which for this case has some nice properties. There's no name confusion because there are no names, it doesn't require defining new types, presenting the signature of the function is almost completely sufficient, needing only a side note specifying which value means what (and only for a couple of cases).
Performing std::get<0>(func(...)) would provide the same result as the trivial operators (+, -, *, /) would in most platforms, performing std::get<1>(func(...)); would recover information lost by the trivial operators (+, -, *, /) (i.e. carry/borrow bits, high bits in mul_wide, and the remainder on division).
While not perfect, it is quite suitable for the intended purpose, without visible undesirable features except in one specific case "mul_wide".

"mul_wide" has to output high and low bits of the resulting multiplication, in platforms that don't have direct hardware support for a specific type but has a wider integer type an implementer may want to implement "mul_wide" by internally casting the inputs to the wider integer, doing a trivial multiplication and then splitting the high and low bits. By defining that the low bits are to be return on the first element of the tuple, on little-endian machines it just so happens that the memory layout matches the fact that low bytes of an integer come first allowing a compiler to optimize away any bit manipulation in all cases. But we need to acknowledge that big-endian machines also exist, and it just so happens that the expected byte order is reversed, a compiler can still optimize away bit manipulation depending on what happens on assignment, but it might not be possible do so in all circumstances if the value needs to be passed to an unknow context (like taking an address). Using a new specialized data structure "mul_wide_result" that only defines that "result_high" and "result_low" must be members of it without specifying the order in which they should appear, an implementer would be free to swap the values around to pick the best memory layout for their specific platform.
In my opinion, considering that big-endian devices are now a days relatively rare, and considering the rare situations where a compiler couldn't just optimize the whole thing anyway, that most use cases are for a widest integer type available on that platform where this trick couldn't be used anyway, and it's just "mul_wide" that has this problem, this shouldn't justify using new data type over a std::tuple.

However, feedback from the initial draft led to the conclusion that tuples are not popular among developers given the ambiguity of the placement of outputs of mul_wide and dive_wide in anonymous structures. And thus, named structures were selected for the sake of improved clarity.

6.4. Extra classifications
With the exception of div_wide, all functions have "well" defined behavior, and they can all be made **constexpr** and **noexcept**. There's a clear benefit to computing things at compile time if possible, and if one can optimize assuming that they don't throw exceptions, then why shouldn't they be?

7. Naming
Names like **add**, **sub**, **mul**, **div**, have long standing unwritten conventions as to what they should mean, its usage is unambiguous, there is precedent for it, and they are all exactly 3 characters long which makes them look really nice when aligning them together. So I propose to keep that. The rest of the text in the function name are mostly plain English and are well know in regards to their meaning.

Some may object to the naming of sub_borrow as opposed to sub_carry, as this may be seen as "an endorsement of intel's naming convention". My counterargument to this objection is "intel is not wrong", in mathematical lingo "borrow" is used for subtraction not "carry", and it makes sense from a clarity perspective in terms of what the flag means. When the flag is 1 "carry" means the flag "adds" to the value, "borrow" means the flag "subtracts", hence add_carry and sub_borrow. Not sub_carry (i.e. flag adds 1 after subtracting), and not add_borrow (i.e. flag subtracts 1 after adding).

The name of the accompanying data structures used for return types will use the name of the function with the post-fix "_result", this is practice that is common in the standard (for example std::from_chars_result). One noticeable exception to this rule is the return structure for div_wide which drops the "_wide", i.e. div_result. The reasoning for this is two fold, first because the name is available, secondly because the returning output of div_wide and potentially extended future version of std::div are exactly the same, and the same type can be used for both without the need to proliferate unnecessary identical structures.

8. Notable objections and FAQ

Q: Why is safe overflow excluded?
A: The [analysis paper] and [P3018R0] suggests the addition of "add_overflow", "sub_overflow", "mul_overflow", and "div_overflow", and yet they are not part of this paper.
This is intentional, after better analysis of the problem, I have concluded that perhaps it is better not to. And here is why:

add_overflow/sub_overflow
add_overflow and sub_overflow, are essentially the same as add_carry/sub_borrow except that the carry/borrow bit are fixed to 0. And indeed, specialized CPU instructions exist to perform this behavior which are different from those required by a generic add_carry/sub_borrow (when the carry/borrow bit aren't known at compile time).
However, I feel that this is best solved with a compiler optimization. If the compiler can deduce at compile time that when add_carry/sub_borrow is used the carry/borrow bit is a constant equal to 0, it is free to decide to use the cheapest instruction, i.e. to degenerate add_carry/sub_borrow to instructions that add_overflow/sub_overflow would have emitted without these functions being explicitly provided.

div_overflow
Unsigned integer division never overflows, and signed integer division only overflows in the degenerate case INT_MAX/-1 which is trivial to check.
One may also note that division by 0 is still undefined, and a user is expected to protect against this case before trying to divide.
In addition, there's no CPU instruction that would give the right result in the degenerate case; a library implementer would always need to explicitly check for the degenerate case before performing the division.
Given these, a user would be better served by being provided with a **is_div_defined** to the check themselves and then decide what behavior their application should have.

mul_overflow
Although CPU instruction support exists in the form of overflow flag on multiplication, and although the behavior makes sense, the use cases are questionable.
One could still implement a sub-optimal "mul_overflow" by using "mul_wide" if need be. But it is still the odd one out, it is better left for a future proposal if a use case is found.

Q: Why does div_wide have a large range of undefined behavior instead of outputing 2 quotient words for high and low bits?
A: To understand this we need to look at the intended usage of the function. I.e. to perform arithmetic on arbitrarily large integers. While the variant of 2 quotient words would be useful to divide a number 2 words wide, in a succinct fashion, its usefulness simply stops when you want to divide a number that has any other number of words.

For example, how would you divide a number with 3 words? You would have to chain multiple division operations. Starting by dividing the "high words" first and then use its remainder (as high bits) and combine it with the yet unused dividend "low bits" into a subsequent division operation to extract the lower bits of the result. On this second division, because the high bits of the dividend are the remainder of a previous division operation by the same divisor (i.e. guarantee that dividend_high < divisor) any theoretical "high bits of a second quotient word" would necessarily be zero (and had they not been zero this operation would need to be more complicated).
How would you divide a number with **N** words? Same idea repeated! Start by dividing the high words first, then use the remainder on the previous operation as the "dividend_high" of the next operation, and keep integrating lower and lower significant bits until all bits have been processed (see 4.3). At no point in the chain would a second "high bits quotient word" would ever not be zero. Had they not been zero, if they existed it would just stand in the way.
Plus, I can get the same result as a "2 quotient" version would by chaining 2 operations, not only that, were a "2 quotient" version ever be designed likely the most efficient way to implement it would be exactly by chaining 2 "single word quotient" divisions as described.
So not only is the proposed version closer to how existing hardware works (or otherwise how a more efficient division algorithm would be written), it also allows a user to explicitly express that a "second high quotient bits" are not expected, and thus write much more efficient software than what would otherwise be possible to express.

Q: Why does the high_bits and low_bits of the signed mul_wide_result are both signed? Especially considering that on the low_bits there are no signed bits.
A: The first step to understand the solution to this problem is to realize that this is not exclusive to mul_wide. Take for example add_carry, when the operands do not cause an overflow, one would simply read the correct signed integer result from `low_bits`. However, if the result would overflow then this is no longer the case, it can't, by definition an overflow means that the value cannot be represented in the available bits, looking at the "sign" bit no longer indicates the correct sign of the actual value. In order to make a correct use of an overflown value, one must treat the resulting object as a bag of bits and only then be able to properly re-interpret the result for what it trully means and act accordingly. For example, if we are only interested in safe-arithmetic then an error handling path can be taken when the overflow bit is set, or if we are using multi-precision we add an extra word to the object's representation.
But couldn't the low_bits of the signed 'mul_wide_result' be unsigned while the high_bits remain signed and thus still accurately represent the result? No. The 'sign bit' is no more represents the sigdness of the number than the other bits represent the "non-signed" part, it's incidental that this works in this way. Take for example -1 which with 8bits is typically represented in binary as 1111'1111 if we remove the sign bit (i.e. 111'1111) this value does not represent an unsigned 1 (which is 127 and not 000'0001). 1111'1111 is only -1 as in so far that is +1 away from 0, and it is done so for implementation convenience, in practice 1111'1111 is only -1 because we mapped its representation that way and because it is easier to design hardware this way. You can't just arbitrarily remove bits and expect integers to retain its intrinsic meaning. It's the complete set of bits that give a concrete meaning to the number being represented, and it is not the collection of its parts. Using unsigned integers to represent the lower bits of the number does not make it a more "accurate" representation, the same way using a signed value for the high_value is not a good representation of the actual value, a (-52; 132) does not make a -13180 anymore than a (-52; -124) does, but bitwise they are identical. The whole point of these functions is because we want to do arithmetic with numbers that cannot be represented by the standard types, but whose exact bitwise content is relevant and useful. By definition, there is no correct representation because it is not representable. That is the whole point of these functions! The only criteria that is relevant is convenience. What is more convenient?
If an unsigned number is used, in all circumstances the types must be interpreted as if they were a bag of bits or casted to something else (if at all possible). If signed is used, in special circumstances (ex. high_bits = 0 and low_bits >= 0; or high_bits = -1 and low_bits < 0) the low_bits represent the exact value of the number for its underlying type, no casting required.
We could have alternatively decided not to provide support for signed integers, and kicked this problem further down the line, but then we would be dropping users who are interested in these facilities to perform safe arithmetic, which includes signed integers; and it is inevitable that we would come to this exact same conclusion. This way is just more convenient. And if you find this disconnect uncomfortable, get used to it! There is not

going to be a "better way", and for users of this facility it is not even a problem. A signed integer has the same number of bits and can represent the same amount of numbers as an unsigned one. We know that the individual words by themselves cannot represent the actual number; but that does not prevent from correctly interpreting the bits as they should and perform the right computation regardless.
Just "shut up and calculate" approach is used.

9. Reference implementation

A [reference implementation] can be found on github. However, it is important to note what this implementation is and what it isn't.
It can be used as a point of reference in order to experiment with the behavior of the features being proposed, and to illustrate how this feature could have looked like had it been implemented as a library without any additional support.
This code is not necessarily bug free. Bust most importantly it is **not** how a final implementation should look like.
It perfectly illustrates the challenges a developer would face if they wanted to implement these features as a portable and independent third-party library.
C++ developers when writing C++ have limited ability to express in code that these functions should use different types of optimizations depending on how the results of these functions are later used, or mutate depending on the properties of the input arguments that the compiler can know at compile time. Meaning that the optimization opportunities referenced in Annex A would be outside of reach for a third-party developer without explicit support of the compiler. And without these optimizations the code generation is so heavily penalized compared to what it could have been as to make this proposal almost pointless.
The reference also illustrates the difficulty of implementing these features effectively using only standard C++, and in order to be efficient it relies heavily on compiler specific features and even inline assembly with instructions specific to a particular CPU. A developer trying to deliver these features as a portable third-party library would be faced with the monumental task of having to tailor code specifically for every permutation of compiler and CPU that the library aimed to support. This would result in a large amount of code that would need to be written and maintained for what is in practice a few lines of assembly.
Hence why this feature should be a part of the standard.

10. Wording

Wording is done in reference to [N5008].

In subclause 26.9 [numeric.ops.overview], change <numeric> as indicated:

```
~~// 26.10.17, saturation arithmetic~~
// 26.10.17, overflow arithmetic
```

In subclause 26.9 [numeric.ops.overview], add to header as indicated:

```
template<class T, class U>
  constexpr T saturate_cast(U x) noexcept;          // freestanding

template<class T>
struct add_carry_result {       // freestanding
    T low_bits;
    bool overflow;
};

template<class T>
using sub_borrow_result = add_carry_result<T>;

template<class T>
struct mul_wide_result {       // freestanding
    T low_bits;
    T high_bits;
};

template<class T>
struct div_result {            // freestanding
    T quotient;
    T remainder;
};

template<class T>
constexpr add_carry_result<T> add_carry(T x, T y, bool carry) noexcept;
//freestanding
template<class T>
constexpr sub_borrow_result<T> sub_borrow(T minuend, T subtrahend, bool
borrow) noexcept;  //freestanding
template<class T>
constexpr mul_wide_result<T> mul_wide(T x, T y) noexcept;
//freestanding
template<class T>
constexpr div_result<T> div_wide(T dividend_high, T dividend_low, T divisor
) noexcept;     //freestanding
}
```

Rename subclause 26.10.17 ~~[numeric.sat]~~ to [numeric.overflow]

Amend subclauses 26.10.17 as follows

~~26.10.17.1 Arithmetic functions [numerics.sat.func]~~

26.10.17.1 Arithmetic typedefs [numerics.overflow.typedefs]

```
template<class T>
struct add_carry_result {
    T low_bits;
    bool overflow;
};
```
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).

```
template<class T>
using sub_borrow_result = add_carry_result;

template<class T>
struct mul_wide_result {
    T low_bits;
    T high_bits;
};
```
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).

```
template<class T>
struct div_result {
    T quotient;
    T remainder;
};
```
Constraints: T is a unsigned integer type ([basic.fundamental]).

26.10.17.2 Arithmetic functions [numerics.overflow.func]

[Note 1: In the following descriptions, an arithmetic operation is performed as a mathematical operation with infinite range and then it is determined whether the mathematical result fits into the result type. — *end note*]

```
template<class T>
  constexpr T add_sat(T x, T y) noexcept;
```
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Returns: If x + y is representable as a value of type T, x + y; otherwise, either the largest or smallest representable value of type T, whichever is closer to the value of x + y.

```
template<class T>
  constexpr T sub_sat(T x, T y) noexcept;
```
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Returns: If x − y is representable as a value of type T, x − y; otherwise, either the largest or smallest representable value of type T, whichever is closer to the value of x − y.

```
template<class T>
  constexpr T mul_sat(T x, T y) noexcept;
```
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Returns: If x × y is representable as a value of type T, x × y; otherwise, either the largest or smallest representable value of type T, whichever is closer to the value of x × y.

```
template<class T>
  constexpr T div_sat(T x, T y) noexcept;
```
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Preconditions: y != 0 is true.
Returns: If T is a signed integer type and x == numeric_limits::min() && y == -1 is true, numeric_limits::max(), otherwise, x / y.
Remarks: A function call expression that violates the precondition in the Preconditions element is not a core constant expression ([expr.const]).

template<class T>
constexpr add_carry_result<T> add_carry(T x, T y, bool carry) noexcept;
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Returns: add_carry_result<T> with the member low_bits set to the result of x + y + (carry ? 1 : 0) truncated to the size of T, the member overflow is set to true if result is not representable as a value of type T and false if otherwise.

template<class T>
constexpr sub_borrow_result<T> sub_borrow(T minuend, T subtrahend, bool borrow) noexcept;
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Returns: sub_borrow_result<T> with the member low_bits set to the result of minuend - subtrahend - (borrow ? 1 : 0) truncated to the size of T, the member overflow is set to true if result is not representable as a value of type T and false if otherwise.

template<class T>
constexpr mul_wide_result<T> mul_wide(T x, T y) noexcept;
Constraints: T is a signed or unsigned integer type ([basic.fundamental]).
Returns: mul_wide_result<T> with the result of x × y, the member low_bits is set to the least significant bits that can fit in a value of type T and the member high_bits is set to the most significant bits.

template<class T>
constexpr div_result<T> div_wide(T dividend_high, T dividend_low, T divisor) noexcept;
Constraints: T is an unsigned integer type ([basic.fundamental]).
Returns: div_result with the member quotient set to the result of (dividend / divisor) where dividend is a number twice the width of T who's high bits are set to dividend_high and low bits are set to dividend_low; and the member remainder set to the remainder of the same division.
Preconditions: (dividend_high < divisor) evaluates to true.
Remarks: A function call expression that violates the precondition in the Preconditions element is not a core constant expression ([expr.const]).

~~26.10.17.2 Casting[numeric.sat.cast]~~
26.10.17.3 Casting[numeric.overflow.cast]
template<class R, class T>
  constexpr R saturate_cast(T x) noexcept;
Constraints: R and T are signed or unsigned integer types ([basic.fundamental]).
Returns: If x is representable as a value of type R, x; otherwise, either the largest or smallest representable value of type R, whichever is closer to the value of x.

Add a feature-test macro in [version.syn]:

#define __cpp_lib_overflow_arithmetic

11. Acknowledgements

Thanks to Jan Schultke and Jonathan Wakely for their editorial reviews.

12. References

1. ISO/IEC JTC1 SC22 WG21 P0543R3: Saturation arithmetic by Jens Maurer
2. ISO/IEC JTC1 SC22 WG21 P3018R0: Low-Level Integer Arithmetic by Andreas Weis
3. Unified integer overflow arithmetic analysis paper
4. Intel® 64 and IA-32 Architectures Software Developer's Manual Vol.2
5. Arm® Architecture Reference Manual
6. MSVC intrinsics
7. Clang builtins
8. Code extract on floating point char conversion
9. https://github.com/tmiguelf/std_prop_overflow - Reference implementation
10. N5008 - Working Draft Programming Languages — C++

Annex A. Code generation and optimization suggestions

It is important to note that none of the functions proposed in this paper are intended to translate into a function call.
Although an actual function (that makes almost no assumptions) may need to exist in case a user might want to take a function pointer (or do memory inspection), the compiler should do a best effort to evaluate the context on how the functions are being used and replace them inline with optimized instructions on an "as-if" principle (unless the platform being developed for doesn't have instructions that could easily achieve the result, and the implementation is so overwhelmingly complicated as to make the cost of function call is negligeable), and need not enforce any memory layout.
The following is a list of optimization suggestions (to be applied only on platforms where applicable and advantageous)

On all functions

- If all inputs are constant values known at compile time, the function shall be eliminated, and the expected output is computed at compile time and used in place.
- If the output is unused the function shall be eliminated
- When chaining multiple operations where the output of one happens to occur precisely in a register where it can serve as the input of the next operation, surperfluous move instructions can be surpressed. When generating fully optimized code, the compiler can re-order instructions (as long as it doesn't violate a dependency chain) in order to minimize surperfluous moves or flag sets.

add_carry

- The addend operands A and B can be freely swapped (take advantage of the commutative property). This can be useful when chaining multiple operations where one (or both) input is in a register that can be used in the addition but not in the trivial order.
- If the input carry is known to have the value 0 at compile time, the compiler can chose an alternative add instruction (if it exists) that does not take a carry as an input.
- If at least one of the addend operands are known at compile time and:
    - If the value is 0, the compiler can choose to directly add the input carry to the remaining operand or chose an alternative conditional increment instruction.
    - If the value is 1, the compiler can choose to swap the carry with this addend
- If the input carry and at least one of the addend operands are known at compile time, the compiler may add them together and:
    - If the result is 0, it shall be a no-op, the remaining operand shall be used as the result and the output overflow is 0

    - If the result is 1, it can be replaced by an increment instruction
    - If the operands are signed and the result is -1, it can be replaced by a decrement instruction
    - If the operands are unsigned and the result would overflow, it shall be a no-op, the remaining operand shall be used as the result and the output overflow is 1
    - If none of the above and the result did not overflow (signed operands), the compiler can chose to add the constant value to the remaining operand using a cheaper add instruction without an input carry.
- If both addend operands are known at compile time, their sum can be computed at compile time, and:
    - If the result is less than max value for the type but still representable, the resulting overflow is 0.
    - If the result is exactly max value for the type, the output overflow is the input carry.
    - If the result is more than max value for the type, the resulting overflow is 1.
    - If the input operands are signed and the result is exactly min -1, output overflow is the negation of the input carry.
    - If the input operands are signed and the result is less than min -1, the resulting overflow is 1.

  The low_bits can be deduced by conditionally incrementing the precomputed result depending on carry.
- If the output overflow value produces no side effects (unread), the carry/overflow flags may not be read.

sub_borrow

- If the input borrow is known to have the value 0 at compile time, the compiler can chose an alternative subtract instruction (if it exists) that does not take a borrow as an input.
- If the subtrahend is known to have the value 0 at compile time, the compiler can choose an alternative conditional decrement instruction.
- If the input borrow and minuend are known at compile time, the compiler may subtract the borrow from the minuend and:
    - If the operands are signed and the result is 0, the low_bits will be equal to -subtrahend and overflow is 1 if and only if subtrahend is equal to min.
    - If the operands are unsigned and the result is max, overflow is 0 and cheaper subtraction instruction maybe used.
- If the input borrow and subtrahend are known at compile time, the compiler may add the borrow to the subtrahend and:
    - If the result is 0, it shall be a no-op, overflow is 0 and low_bits is equal to minuend.
    - If none of the above and the result did not overflow, the compiler can chose to subtract the constant value to the remaining operand using a cheaper sub instruction without an input borrow.
- If the minuend and subtrahend operands are known at compile time, their difference can be computed at compile time, and:
    - If the result is more than min value for the type but still representable, the resulting overflow is 0.
    - If the result is exactly min value for the type, the output overflow is the input borrow.
    - If the result is less than min value for the type, the resulting overflow is 1.
    - If the result is exactly max +1, output overflow is the negation of the input borrow.
    - If the result is more than max +1, the resulting overflow is 1.

  The low_bits can be deduced by conditionally decrementing the precomputed result depending on borrow.
- If the output overflow value produces no side effects (unread), the carry/overflow flag may not be read.

mul_wide

- The operands A and B can be freely swapped (take advantage of the commutative property). This can be useful when chaining multiple operations where one (or both) input is in a register that can be used in the multiplication but not in the trivial order.
- If one of the operands is known at compile time and:
    - The value is 0, it shall be a no-op, the output shall have 0 on high_bits and low_bits.
    - The value is 1, it shall be a no-op, the output shall have 0 on high_bits, and low_bits shall be equal to the remaining input
    - The input operands are unsigned and the value is a power of 2, it can be replaced by a left shift instruction (provided that the ejects bits are recovered to form the high_bits)

    - The input operands are unsigned and the value is the max representable value, in the condition that it would be cheaper to do so, the result can be computed by having low_bits equal minus the remaining input, and high_bits is equal to remaining input -1 if remaining input is not 0 (otherwise 0)
    - The input operands are unsigned and the value is -1, low_bits shall be equal to minus the remaining operand and high_bits shall be 0 or 1 filled depending if the remaining operand is negative.
- If the resulting high_bits are used but known at compile time to be only compared against 0, a cheaper multiplication instruction may be used and the overflow flag used in place of high_bits. Note: If the optimization where 1 of the operands is known and is the maximum representable value and the operand is unsigned is taken, comparing the remaining operand to be > 1 can be used instead. If the optimization where 1 of the operands is known and is signed and the value is -1 is taken, checking if the remaining operand is not negative can be used instead.
- If the resulting high_bits are unused, cheaper multiplication instruction may be used where high_bits are unretrieved.
- If the resulting low_bits are unused, cheaper instruction can be used to only compute the high_bits of the multiplication.

div_wide
Note: compilation should always fail in a constexpr context if inputs are outside of the defined domain.

- Runtime behavior when inputs are in the undefined domain need not be consistent in any way, they may even display different behavior in seemingly identical circumstances. The implementation may choose to crash the application or return any result, but should not lock the application in an infinite loop.
- If dividend_high is known at compile time and its value is 0, a cheaper division instruction/algorithm without high bits may be used.
- If dividend_high and dividend_low are known at compile time and are both 0, it shall be a no-op, quotient and remainder shall be 0
- If divisor is known at compile time and:
    - If its value is 0, it shall be a no-op, any valid undefined behavior is chosen.
    - If inputs are unsigned and dividend_high is known at compile time, and dividend_high >= divisor, it shall be a no-op, any valid undefined behavior is chosen.
    - If its value is 1, it shall be a no-op, quotient is dividend_low and remainder is 0.
    - If input operands are unsigned and the value is a power of 2, it can be replaced by a right shift instruction (provided that the ejected bits are recovered to form the remainder).
    - If otherwise, traditional known constant division optimization maybe used.

  Note: If div_wide is unguarded and compiler knows at compile time that inputs to div_wide are in the undefined domain, it is strongly advised that compiler issues a warning.

Note: this list is not exhaustive, and many more optimization opportunities may exist depending on the platform.

Annex B. Suggested future work

B.1. div

Suggested signature change (future work)

```
template<class T>
constexpr div_result<T> div(T dividend, T divisor) noexcept;
```

Performs a fused division with remainder.
std::div already exists in the standard. This paper only notes insufficient design and proposes a signature change in future development.

B.2. is_div_wide_defined

Suggested signature

```
template<class T>
constexpr bool is_div_wide_defined(T dividend_high, T dividend_low, T
divisor) noexcept;
```

Description:
Checks if std::div_wide is defined for the input arguments
i.e. Checks that the divisor is not 0, and that result would not overflow

Possible implementation:

```
template<typename T>
[[nodiscard]] constexpr bool is_div_wide_defined(T const hi_dividend,
[[maybe_unused]] T const low_dividend, T const divisor) noexcept
{
    if constexpr(std::is_signed_v<T>)
    {
        using uint_t = std::make_unsigned_t<T>;
        constexpr uintptr_t sign_offset = (sizeof(uint_t) * 8) - 1;
        constexpr uint_t lower_mask = static_cast<uint_t>(~(uint_t{1} <<
sign_offset));

        uint_t const hi  = std::bit_cast<uint_t>(hi_dividend);
        uint_t const low = std::bit_cast<uint_t>(low_dividend);

        uint_t const div = std::bit_cast<uint_t>(divisor);
        uint_t const hi_flag = static_cast<uint_t>((hi << 1) | (low >>
sign_offset));

        if(hi_dividend < 0)
        {
            if(divisor < 0)
            {
                return
                    (hi_flag > div) ||
                    ((hi_flag == div) &&
                        (low & lower_mask));
            }
            else
            {
                uint_t const mirror = ~div;

                return
                    (hi_flag > mirror) ||
                    ((hi_flag == mirror) &&
                        ((low & lower_mask) > ((mirror & lower_mask) +
1)));
            }
        }
        else
        {
            if(divisor < 0)
            {
                uint_t const mirror = (~div) + 1;

                return
                    (hi_flag < mirror) ||
                    ((hi_flag == mirror) &&
                        ((low & lower_mask) < mirror));
            }
            else
            {
                return hi_flag < div;
```

```
                return hi_.lag < div;
            }
        }

    }
    else
    {
        return hi_dividend < divisor;
    }
}
```